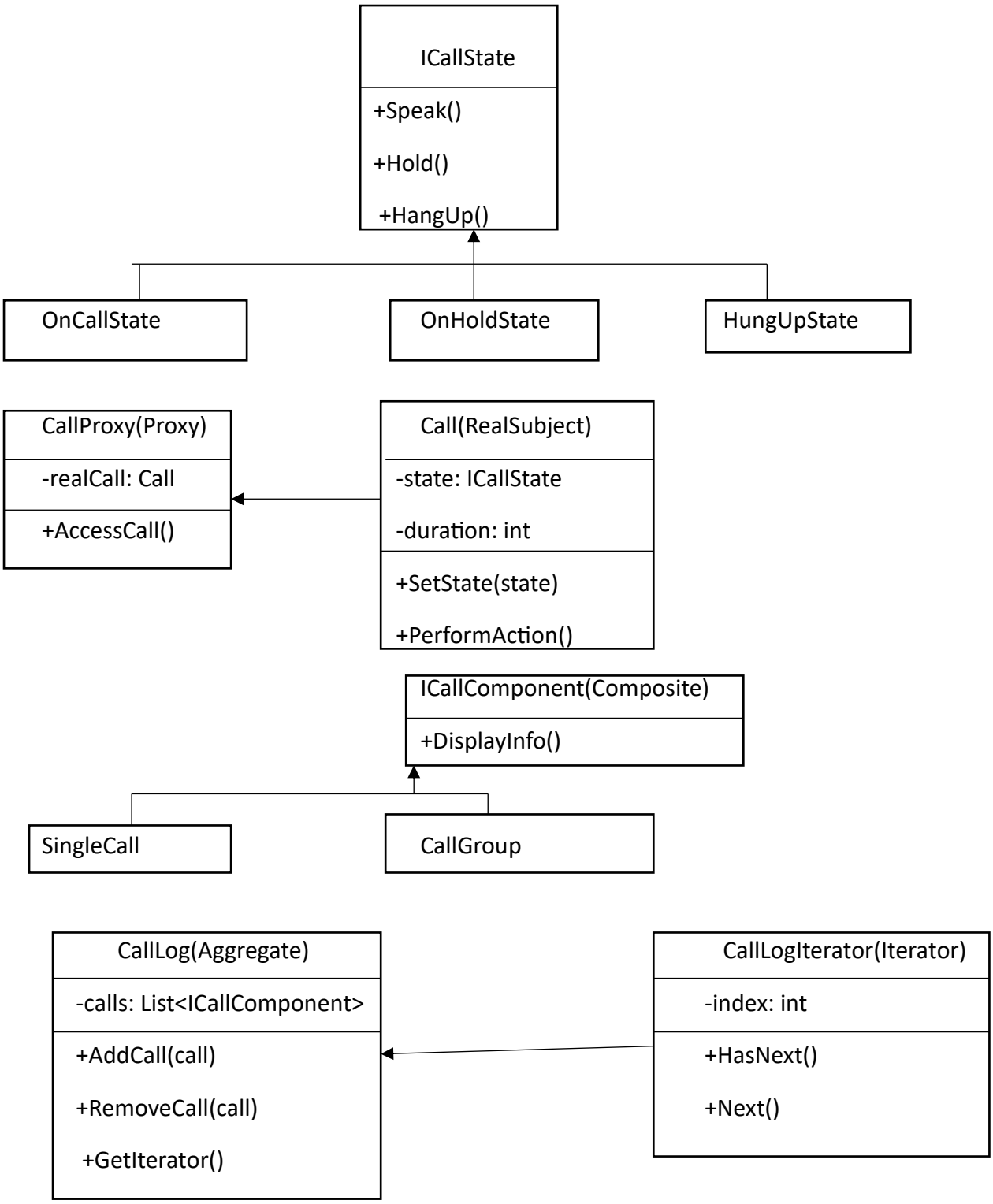
Motivation for Design Patterns

Here's how each required pattern fits the call center scenario:

- **State Pattern**
    - Motivation: Calls can be in different states (OnCall, OnHold, HungUp). Each state changes behavior (speaking allowed, duration recorded, etc.).
    - Classes: Call (Context), ICallState (Interface), OnCallState, OnHoldState, HungUpState.
- **Iterator Pattern**
    - Motivation: The manager needs to move through the virtual call log to find a specific call. Iterator provides sequential access without exposing internal structure.
    - Classes: CallLogIterator, CallLog.
- **Composite Pattern**
    - Motivation: Calls can be grouped (e.g., multiple calls from the same student, or calls grouped by technician). Composite allows treating individual calls and groups uniformly.
    - Classes: ICallComponent (Interface), SingleCall, CallGroup.
- **Proxy Pattern**
    - Motivation: The manager accesses calls indirectly through the virtual call log. The proxy controls access, adds security, or lazy-loads call details.
    - Classes: CallProxy (acts as a stand-in for real Call objects, fetching details only when needed).


ICallState
+Speak()
+Hold()
+HangUp()
OnCallState
OnHoldState
HungUpState
CallProxy(Proxy)
-realCall: Call
+AccessCall()
Call(RealSubject)
-state: ICallState
-duration: int
+SetState(state)
+PerformAction()
ICallComponent(Composite)
+DisplayInfo()
SingleCall
CallGroup
CallLog(Aggregate)
-calls: List<ICallComponent>
+AddCall(call)
+RemoveCall(call)
+GetIterator()
CallLogIterator(Iterator)
-index: int
+HasNext()
+Next()